

Guide - Requetes SQL dans JavaFX

Saisir et executer des requetes SQL via une interface graphique

Document pedagogique - BTS SIO

1. Presentation de l'application

L'objectif est de creer une interface JavaFX qui permet a l'utilisateur de taper une requete SQL dans un champ texte, de cliquer sur un bouton Executer, et de voir le resultat s'afficher dans un tableau dynamique.

L'interface est composee de 3 elements :

- Un champ texte (TextField) pour saisir la requete SQL
- Un bouton Executer pour lancer la requete
- Un TableView dynamique qui s'adapte automatiquement aux colonnes du resultat
- Un label rouge pour afficher les erreurs SQL

■ Contrairement au TableView fixe vu precedemment, ici le tableau cree ses colonnes automatiquement selon les colonnes retournees par la requete SQL.

2. Fichiers du projet

Ce projet necessite seulement 2 fichiers (Film.java n'est plus necessaire) :

Fichier	Role
hello-view.fxml	Interface graphique : TextField, Button, TableView
HelloController.java	Logique : connexion MySQL et execution des requetes
HelloApplication.java	Point d'entree de l'application (inchange)

3. La Vue - hello-view.fxml

Le fichier FXML definit l'interface. On utilise un VBox (disposition verticale) contenant un HBox (disposition horizontale) pour le champ texte et le bouton.

Explication des composants :

Composant	fx:id / attribut	Role
TextField	txtRequete	Champ de saisie de la requete SQL
Button	onAction	Appelle la methode executerRequete() du controleur

Label	labelErreur	Affiche les erreurs en rouge
TableView	tableResultat	Tableau dynamique pour afficher les resultats

Code complet de hello-view.fxml :

```
<?xml version="1.0" encoding="UTF-8"?>
<?import javafx.scene.control.*?>
<?import javafx.scene.layout.*?>
<VBox xmlns:fx="http://javafx.com/fxml"
fx:controller="com.example.comentrejevasql.HelloController"
spacing="10" style="-fx-padding: 20;">
<HBox spacing="10">
<TextField fx:id="txtRequete" prefWidth="500"
promptText="Entrez votre requete SQL ici..."/>
<Button text="Executer" onAction="#executerRequete"/>
</HBox>
<Label fx:id="labelErreur" style="-fx-text-fill: red;"/>
<TableView fx:id="tableResultat" prefHeight="400" prefWidth="700">
</TableView>
</VBox>
```

4. Le Controleur - HelloController.java

Le controleur contient toute la logique. La methode principale est executerRequete() qui est appelee quand l'utilisateur clique sur le bouton.

4.1 Les variables @FXML

On lie les composants FXML aux variables Java avec @FXML. Le TableView utilise un type special : ObservableList<String> car on ne connaît pas a l'avance le nombre de colonnes.

```
@FXML private TextField txtRequete;
@FXML private TableView<ObservableList<String>> tableResultat;
@FXML private Label labelErreur;
```

4.2 La methode executerRequete()

Cette methode fait 4 choses dans l'ordre :

- 1. Recupere le texte saisi dans le TextField
- 2. Reinitialise le tableau et efface les anciennes colonnes
- 3. Execute la requete SQL et recupere les resultats
- 4. Cree les colonnes dynamiquement et remplit le tableau

4.3 Colonnes dynamiques avec ResultSetMetaData

La classe ResultSetMetaData permet de connaître les colonnes du resultat a la volée, sans les connaître a l'avance. On l'utilise pour créer automatiquement une TableColumn par colonne retournée.

```
ResultSetMetaData meta = rs.getMetaData();
int nbColonnes = meta.getColumnCount();
for (int i = 1; i <= nbColonnes; i++) {
    final int index = i - 1;
    TableColumn<ObservableList<String>, String> col =
```

```

new TableColumn<>(meta.getColumnName(i));
col.setCellValueFactory(data ->
new SimpleStringProperty(data.getValue().get(index)));
tableResultat.getColumns().add(col);
}

```

■ La variable `index` doit être `final` car elle est utilisée dans une lambda. Sans `final`, Java ne peut pas la capturer dans la fonction fléchée.

4.4 Code complet de HelloController.java

```

package com.example.comentrejdbc;
import javafx.beans.property.SimpleStringProperty;
import javafx.collections.FXCollections;
import javafx.collections.ObservableList;
import javafx.fxml.FXML;
import javafx.scene.control.*;
import java.sql.*;

public class HelloController {
    @FXML private TextField txtRequete;
    @FXML private TableView<ObservableList<String>> tableResultat;
    @FXML private Label labelErreur;

    private Connection getConnexion() throws SQLException {
        return DriverManager.getConnection(
            "jdbc:mysql://localhost:3306/cinefilm?useSSL=false",
            "root", ""
        );
    }

    @FXML
    public void executerRequete() {
        String requete = txtRequete.getText().trim();
        labelErreur.setText("");
        tableResultat.getColumns().clear();
        tableResultat.getItems().clear();
        if (requete.isEmpty()) {
            labelErreur.setText("Veuillez entrer une requete SQL.");
            return;
        }
        try {
            Connection conn = getConnexion();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(requete);
            ResultSetMetaData meta = rs.getMetaData();
            int nbColonnes = meta.getColumnCount();
            // Creer les colonnes dynamiquement
            for (int i = 1; i <= nbColonnes; i++) {
                final int index = i - 1;
                TableColumn<ObservableList<String>, String> col =
                    new TableColumn<>(meta.getColumnName(i));
                col.setCellValueFactory(data ->
                    new SimpleStringProperty(data.getValue().get(index)));
                tableResultat.getColumns().add(col);
            }
            // Remplir les lignes

```

```
ObservableList<ObservableList<String>> data =
FXCollections.observableArrayList();
while (rs.next()) {
ObservableList<String> ligne = FXCollections.observableArrayList();
for (int i = 1; i <= nbColonnes; i++) {
String val = rs.getString(i);
ligne.add(val != null ? val : "NULL");
}
data.add(ligne);
}
tableResultat.setItems(data);
conn.close();
} catch (SQLException e) {
labelErreur.setText("Erreur SQL : " + e.getMessage());
}
}
}
```

5. Connexion en local avec WAMP

Quand on travaille en local avec WAMP au lieu du serveur du lycee, il faut modifier l'URL de connexion :

Situation	URL de connexion	Mot de passe
Serveur lycee	jdbc:mysql://172.20.0.101:3306/cinefilm	"root"
WAMP en local	jdbc:mysql://localhost:3306/cinefilm	"" (vide par default)

■ Sur WAMP, le mot de passe root est vide par default. Si la connexion echoue, essaie avec "" comme mot de passe.

Ajoute aussi le parametre useSSL=false a la fin de l'URL pour eviter les avertissements SSL :

```
"jdbc:mysql://localhost:3306/cinefilm?useSSL=false"
```

6. Erreurs courantes et solutions

Erreur	Cause	Solution
Communications link failure	Mauvaise IP ou port bloque	Verifier l'IP, utiliser localhost en local
No suitable driver found	Driver MySQL absent du pom.xml	Ajouter mysql-connector-j dans <dependencies>
Tableau vide sans erreur	Requete incorrecte ou table vide	Verifier la requete dans Adminer d'abord
Erreur SQL dans le label rouge	Syntaxe SQL incorrecte	Corriger la requete (ex: FROM au lieu de form)
NullPointerException	fx:id ne correspond pas au nom de variable	Verifier que fx:id = nom de la variable Java

7. Exemples de requetes SQL a tester

Voici quelques requetes a tester dans l'interface :

Requete SQL	Description
SELECT * FROM film	Affiche tous les films
SELECT title, year FROM film	Affiche seulement le titre et l'annee
SELECT * FROM film WHERE year > 1990	Films sortis apres 1990
SELECT * FROM film ORDER BY year DESC	Films tries par annee decroissante
SELECT director, COUNT(*) FROM film GROUP BY director	Nombre de films par realisateur

■ Toutes ces requetes fonctionnent directement dans le champ texte de l'interface sans modifier le code Java.